SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA0302	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	D. Sindhu	**Reg. No.:**	192424197
Section / Batch:	2024	**Date:**	13.07.2026

Code of Conduct

I _____D. Sindhu (192424197)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.
Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Resume Information Extraction using Regular Expressions

Aim

To create a Python program using Regular Expressions (Regex) to extract candidate details from resumes and find eligible candidates.

Procedure

Step 1: Import the re module and create sample resume data.

Step 2: Use Regular Expressions to extract the name, email, phone number, skills, and experience.

Step 3: Display the extracted information for each candidate.

Step 4: Check whether the candidate has **Python skill** and **2 or more years of experience**.

Step 5: Display the names of eligible candidates.

Program

```
import re

# Sample resumes
resumes = [
"""
Name: John Smith
Email: john@gmail.com
Phone: 9876543210
Skills: Python, Java, SQL
Experience: 3 years
""",

"""
Name: Priya
Email: priya@yahoo.com
Phone: 9123456789
Skills: Java, SQL
Experience: 1 year
"""
]

skills_list = ["Python", "Java", "SQL", "Machine Learning", "NLP"]

print("Resume Information")

for resume in resumes:

    name = re.search(r"Name:\s*(.*)", resume).group(1)
    email = re.search(r"\S+@\S+", resume).group()
    phone = re.search(r"\d{10}", resume).group()
    exp = int(re.search(r"\d+", re.search(r"Experience:.*", resume).group()).group())

    skills = []
    for skill in skills_list:
        if re.search(skill, resume, re.IGNORECASE):
```

```
            skills.append(skill)

    print("\nName:", name)
    print("Email:", email)
    print("Phone:", phone)
    print("Skills:", ", ".join(skills))
    print("Experience:", exp, "Years")

    if exp >= 2 and "Python" in skills:
        print("Status: Eligible")
    else:
        print("Status: Not Eligible")
```

Output

```
Resume Information

Name: John Smith
Email: john@gmail.com
Phone: 9876543210
Skills: Python, Java, SQL
Experience: 3 Years
Status: Eligible

Name: Priya
Email: priya@yahoo.com
Phone: 9123456789
Skills: Java, SQL
Experience: 1 Years
Status: Not Eligible
```

Result

The program successfully extracted candidate details using Regular Expressions and displayed the eligible candidates based on Python skill and minimum 2 years of experience.

2. Product Search System using Regular Expressions

Aim

To develop a Python program using Regular Expressions (Regex) to search product names using different search methods.

Procedure

Step 1: Import the re module and create a list of product names.

Step 2: Get the search keyword from the user.

Step 3: Use Regular Expressions to perform exact, prefix, suffix, partial, and case-insensitive searches.

Step 4: Display all matching product names.

Step 5: Display the total number of matching products for each search.

Program

```
import re

# Product list
products = [
    "Laptop",
    "Laptop Bag",
    "Gaming Laptop",
    "Mouse",
    "Wireless Mouse",
    "Keyboard",
    "Mechanical Keyboard",
    "Smart Phone",
    "Phone Charger",
    "Bluetooth Speaker"
]

keyword = input("Enter search keyword: ")

# Exact Search
exact = [p for p in products if re.fullmatch(keyword, p)]

# Prefix Search
prefix = [p for p in products if re.match(keyword, p)]

# Suffix Search
suffix = [p for p in products if re.search(keyword + r"$", p)]

# Partial Search
partial = [p for p in products if re.search(keyword, p)]

# Case-Insensitive Search
ignore_case = [p for p in products if re.search(keyword, p, re.IGNORECASE)]

print("\nExact Search:", exact)
print("Total:", len(exact))

print("\nPrefix Search:", prefix)
print("Total:", len(prefix))

print("\nSuffix Search:", suffix)
print("Total:", len(suffix))

print("\nPartial Search:", partial)
print("Total:", len(partial))

print("\nCase-Insensitive Search:", ignore_case)
print("Total:", len(ignore_case))
```

Output

```
Enter search keyword: Laptop

Exact Search: ['Laptop']
Total: 1

Prefix Search: ['Laptop', 'Laptop Bag']
Total: 2

Suffix Search: ['Laptop', 'Gaming Laptop']
Total: 2

Partial Search: ['Laptop', 'Laptop Bag', 'Gaming Laptop']
Total: 3

Case-Insensitive Search: ['Laptop', 'Laptop Bag', 'Gaming Laptop']
Total: 3
```

Result

The Python program successfully searched products using exact, prefix, suffix, partial, and case-insensitive Regular Expression searches. It displayed all matching product names and the total number of matching products for each search.

3. University Registration System using Regular Expressions

Aim

To develop a Python program using Regular Expressions (Regex) to validate student registration details and generate the registration status.

Procedure

Step 1: Import the re module and get the student details as input.

Step 2: Use Regular Expressions to validate the register number, email, course code, semester, and mobile number.

Step 3: Display a valid or invalid message for each field.

Step 4: Check whether all the details are valid.

Step 5: Display the final registration status.

Program

```
import re

# Get input
reg_no = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")

# Register Number Validation (Example: 23IT101)
if re.fullmatch(r"\d{2}[A-Z]{2}\d{3}", reg_no):
    print("Register Number: Valid")
    reg_valid = True
else:
    print("Register Number: Invalid")
    reg_valid = False

# Email Validation (Example: student@university.edu)
if re.fullmatch(r"[a-zA-Z0-9._%+-]+@university\.edu", email):
    print("Email: Valid")
    email_valid = True
else:
    print("Email: Invalid")
    email_valid = False

# Course Code Validation (Example: CS101)
if re.fullmatch(r"[A-Z]{2,3}\d{3}", course):
    print("Course Code: Valid")
    course_valid = True
else:
    print("Course Code: Invalid")
    course_valid = False

# Semester Validation (1 to 8)
if re.fullmatch(r"[1-8]", semester):
    print("Semester: Valid")
    sem_valid = True
else:
    print("Semester: Invalid")
    sem_valid = False

# Mobile Number Validation
if re.fullmatch(r"[6-9]\d{9}", mobile):
    print("Mobile Number: Valid")
    mobile_valid = True
else:
    print("Mobile Number: Invalid")
    mobile_valid = False

# Final Status
if reg_valid and email_valid and course_valid and sem_valid and mobile_valid:
    print("\nRegistration Status: Successful")
else:
    print("\nRegistration Status: Failed")
```

Output

```
Enter Register Number: 23IT101
Enter Institutional Email: student@university.edu
Enter Course Code: CS1A1
Enter Semester: 5
Enter Mobile Number: 9876543210
Register Number: Valid
Email: Valid
Course Code: Invalid
Semester: Valid
Mobile Number: Valid

Registration Status: Failed
```

Result

The Python program successfully validated the register number, institutional email address, course code, semester, and mobile number using Regular Expressions. It displayed validation messages for each field and generated the final registration status as Successful or Failed.